EE2026 FPGA Pinball

Code, concepts and rebuilding guide

Understand the current circuit. Learn the digital-design principles. Build a new interactive FPGA project.



Basys 3 / Artix-7 | OLED 96 x 64 on JC | Native VGA 800 x 600

Verified implementation snapshot: 14 October 2026. Detailed explanations, worked examples, source map, exercises, timing and debugging workflow.

Contents

1. How to read and use this guide 4

2. What the complete machine does 4

3. The EE2026 knowledge behind the project 6

4. Digital-design foundations before reading the source 7

5. Architecture, ownership and the module connections 8

6. Clocks, resets, physical inputs and the mouse 10

7. From controls to game rules 11

8. The five-state controller and event priorities 12

9. Charge, lives and protected relaunches 13

10. Contact masks, combos and collection rewards 14

11. Half-point scoring and the three-edge pipeline 15

12. Sequential reasoning and a rebuilding exercise 15

13. Motion is a sampled calculation, not a moving pixel 16

14. Contacts: approximating useful geometry in hardware 17

15. Sharing one collision engine between two balls 19

16. Recreate the physics incrementally 21

17. How one game produces two different pictures 21

18. Procedural drawing and layer priority 22

19. Coordinates: logical playfield versus native VGA 22

20. Colours and small font memories 23

21. OLED scan and the supplied SPI driver 23

22. VGA raster timing and the pixel clock 23

23. Stable pictures: producer registers and frame snapshots 24

24. Effects that consume little state 25

25. Build your own display system in manageable steps 25

26. Reading the source efficiently 26

27. Build a new game in stages 26

28. Verification: prove contracts with self-checking tests 28

29. Vivado flow, constraints and resource reasoning 29

30. Build, inspect and program this exact project 30

31. Debug symptoms by tracing the responsible contract 31

32. Extending a finalized design without breaking its contracts 32

33. Questions you should be able to answer and explain 32

34. Glossary and source references 33

A teaching guide to the finalized Basys 3 project. Written for someone who did not write the code and wants to understand it well enough to design another FPGA game.

Design snapshot: 14 October 2026, `xc7a35tcpg236-1`, Vivado 2025.2. Simultaneous 96 x 64 color OLED on JC and native 800 x 600 VGA. One/two-ball selection is in the menu; SW1 is ignored. The optional mouse uses SW0. The authoritative implementation evidence is `build/manifest.json` and `docs/VALIDATION.md`.

This document explains the actual RTL, then turns its design choices into reusable lessons. Small teaching snippets are marked as examples; they are not replacements for the complete modules. A section called "rebuild" describes a new implementation you can develop yourself, rather than instructions to paste this entire project into a new folder.

1. How to read and use this guide

Start with the architecture and digital-design foundations before reading the arithmetic-heavy collision code. Then follow one action through the design: press Center, charge, release, move a ball, collide, score, and draw the next frame. That path reveals why the modules exist and which signals connect them.

On a second pass, open each named source file beside its explanation. All custom RTL lives in `FDP2026.srcs/sources_1/new/`. The top file contains three modules, so "eleven custom RTL files" means thirteen custom modules. The supplied Verilog OLED interface and two VHDL mouse files add three more modules. They are preserved byte-for-byte.

Use three levels of understanding. First, know what each module promises at its ports. Second, know when its registers change. Third, know how the combinational logic computes the next value. Understanding only the pictures or only the equations is insufficient: a correct FPGA system needs the right values on the right clock edges.

After each major topic, explain it aloud without referring to the source. For example: why does one button press create one menu event, while a held button continuously operates a flipper? Why do two balls share a collision circuit? Why does a graphics frame contain a copy of the game state? These questions test the design principles you will need in a new project.

The original proposal separates motion, collisions, graphics, and game/input. The final design follows that decomposition and adds OLED output, moving objects, two-ball play, reduced scoring, richer feedback, and original reaction graphics. Scheduling and member-allocation text in the proposal are historical context, not descriptions of a different current circuit.

2. What the complete machine does

This project implements a pinball game as interconnected digital circuits. It does not run a C game loop on a processor. The FPGA stores positions, velocities, game state, score, timers and input history in registers. Arithmetic and comparison circuits continuously calculate candidate results. Clock edges decide when those candidates become the new state.

The table has three bumpers, two side targets, three pass-through rollover lanes, a spinner, two diamond slings, three stand-up targets, walls, guides, a shooter lane, and two flippers. Two bumpers move on short tracks. A game has three lives, with finite launch/target saves; the turn continues until all selected balls drain. Both displays show the same game, although they do not necessarily start their frames at the same instant.

Board controls cover all essential actions. The mouse is an additional input device. The VGA connector carries RGB and synchronization signals; an external active VGA-to-HDMI converter produces HDMI. The FPGA does not generate a native HDMI/TMDS signal in this project.



Complete game on the OLED and native VGA

The OLED portion of this preview is enlarged for readability. The native VGA renderer computes its own fine pixels, text and shapes. A visual preview is evidence of renderer output, not proof that a physical converter or OLED has been tested.

Hardware and current measured budget

Item	Current design
FPGA target	Artix-7 `xc7a35tcpg236-1` on Basys 3
Main clock	100 MHz, 10 ns period
Physics enable	Nominal 240 updates/second, generated every 416,666 main clocks
OLED interface clock	6.25 MHz through a global clock buffer
VGA pixel clock	40 MHz from an MMCM and global buffer
OLED connector	JC, RGB565, 96 x 64 pixels
VGA active area	800 x 600 pixels, RGB444 on board pins
Routed LUTs	18,380 / 20,800 = 88.37%
Routed flip-flops/registers	3,563 / 41,600 = 8.56%
DSP blocks	54 / 90 = 60%
Block RAM tiles	0 / 50
Worst routed setup/hold slack	+0.380 ns / +0.052 ns

These are different resource categories. "88.37% of logic gates" is an imprecise description: 88.37% is the measured LUT utilization. Low register utilization does not imply that another large arithmetic or graphics feature will fit. A LUT

implements a small Boolean lookup table; a register remembers state; a DSP implements dedicated arithmetic; block RAM stores larger memories. A design can exhaust one category while leaving others available.

The hardware roles and USB-to-PS/2 mouse bridge are described in the Digilent Basys 3 reference manual. The exact counts and design timing above come from this project's routed reports, not a board marketing figure.

3. The EE2026 knowledge behind the project

The local EE2026 Lab 4 manual introduces sequential circuits, clocks, counters, behavioural modelling and structural composition. The project manual adds OLED coordinates, RGB color, input handling, smooth movement, seven-segment multiplexing, and integration into one bitstream. Pinball combines these foundations into a larger system.

EE2026 concept	Where to observe it	What to understand
Boolean/dataflow logic	Renderer shape tests, input predicates, hit masks	A condition describes a combinational circuit
Structural modelling	`My_Top_Module` instances and port connections	Modules are hardware components connected by nets
Sequential circuits	Game state, input counters, physics registers	New state depends on previous state and current inputs
Binary counters	Tick counter, debounce counters, animation/timers	Width and terminal-count conditions determine behaviour
Clock division	OLED divider, MMCM pixel clock	Frequency division differs from a clock-enable pulse
Finite-state machines	MENU/READY/PLAY/PAUSE/OVER; driver startup	State determines legal transitions and operations
Multiplexers	Shared collision operands, selected font/glyph, scan digit	Selection shares hardware while requiring clear timing
Number representation	Signed Q8 motion; BCD score	The bit pattern and its interpretation must agree
Display interfacing	OLED SPI, VGA raster, seven-segment anodes	Pixels or digits require an interface protocol
System integration	Hit events, commands, scene snapshots	A common signal contract prevents module conflicts
Verification	Self-checking benches and rendered frames	Expectations must be checked, not merely observed
FPGA implementation	Synthesis, placement, routing, timing constraints	Correct simulation does not establish physical timing

This is a learning map, not a promise of marks or a replacement for current assessment instructions. The game uses a 5 ms stable-sample input filter for responsiveness. The project manual's introductory exercise describes a different debounce strategy with a longer lockout window. Learn the shared principle-one physical press should create one event-without assuming the game's constants are the required values for every lab exercise.

The proposal's four functional components are visible here: physics calculates motion, collisions correct it, rules interpret contact events, and graphics displays the resulting state. Input processing and the top-level wiring connect those responsibilities. Being able to explain this separation is more useful than memorizing individual Verilog lines.

4. Digital-design foundations before reading the source

Combinational logic versus stored state

A combinational circuit produces an output from its current inputs without remembering previous values. An `assign` expression, comparison or fully assigned `always @*` block can describe such logic. In a renderer, the pixel at `(x,y)` is determined from the coordinates and the selected scene. No sequence of software drawing calls is required.

A clocked `always @(posedge clk)` block describes state updated at clock edges. `position_x`, `score_bcd`, `state`, debounce counts and timers are examples of remembered values. A `reg` declaration by itself does not guarantee a physical flip-flop: the assignment context determines whether synthesis infers combinational logic, a flip-flop, or accidentally a latch.

Teaching example, not a drop-in project module:

```
// Combinational condition: a rectangle exists at these coordinates.
wire inside_box = x >= 10 && x < 20 && y >= 8 && y < 16;

// Sequential state: move only on a selected update edge.
always @(posedge clk) begin
    if (reset)
        object_x <= 7'd10;
    else if (step_enable)
        object_x <= object_x + 1'b1;
end
```

All module instances operate concurrently. A source file's ordering does not mean the FPGA finishes input processing, then runs rules, then runs graphics as separate software tasks. Nevertheless, register boundaries introduce cycle latency. One module may consume a command on the following clock because nonblocking assignments publish it after the current edge.

Why assignment type matters

Clocked state in this project uses nonblocking assignments, `<=`. The right-hand sides use values from before the edge, and the left-hand sides update together. Combinational calculation uses blocking assignments, `=`, so a later expression in the block can use a temporary result computed earlier in that block.

Consider this teaching example:

```
always @(posedge clk) begin
    a <= b;
    c <= a;
end
```

After an edge, `a` holds the old `b`, while `c` holds the old `a`. This is a two-register pipeline; `c` does not immediately receive the new `b`. The game's award pipeline and delayed mouse-event handling depend on this distinction.

Within one clocked block, multiple active nonblocking assignments to the same register are resolved by their procedural order: the last executed assignment wins. The game intentionally puts reset/new-game/drain and collection updates in particular places. Reordering code for appearance can change a simultaneous-event outcome.

In a combinational block, give every result a default before conditional overrides. If some path fails to assign a result, the result must retain its previous value, so synthesis may infer a latch. A latch can be useful in a deliberate design, but these custom combinational modules do not intend to infer one.

Width, signedness and safe arithmetic

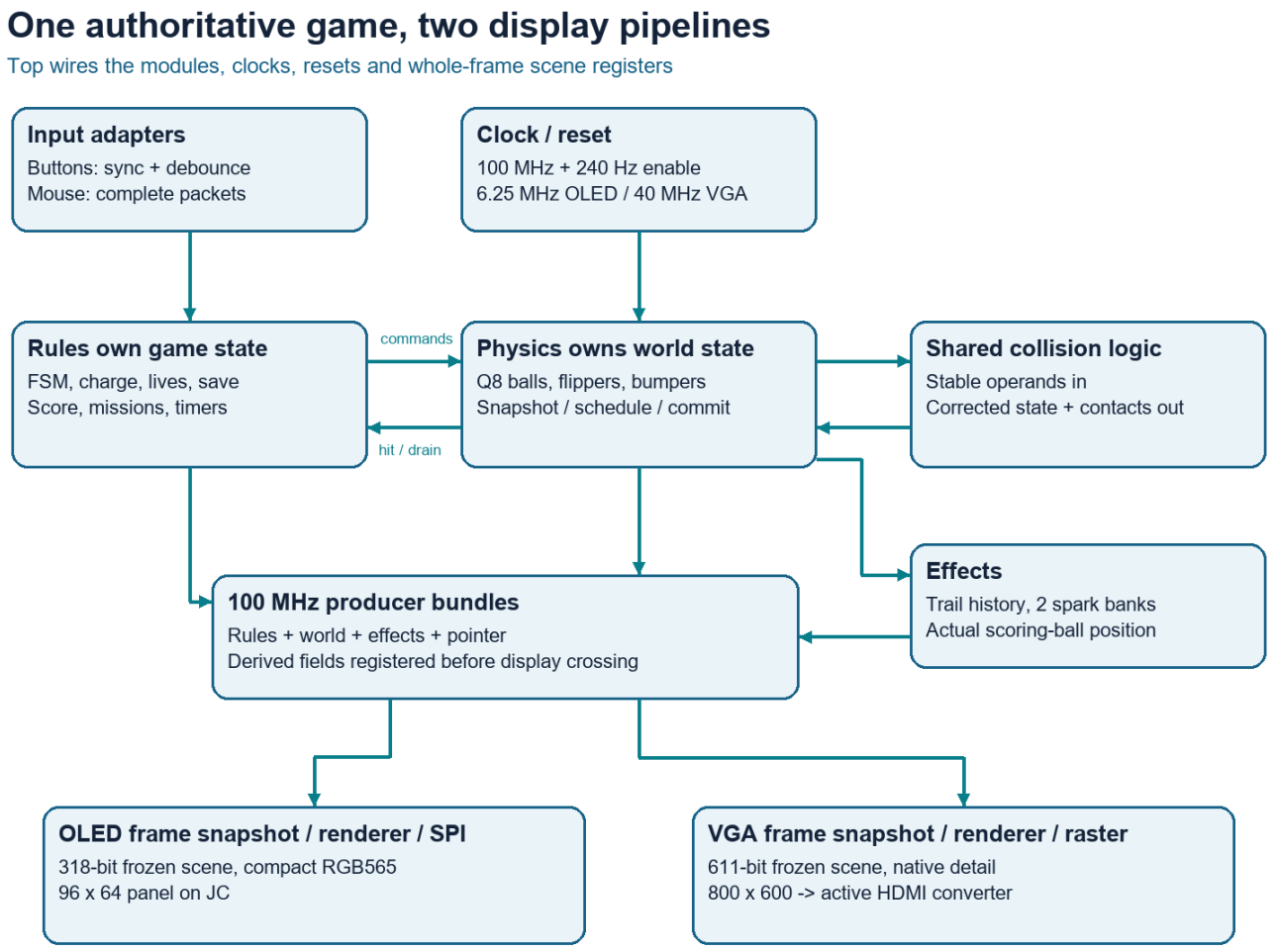
Hardware numbers have finite widths. Seven unsigned bits represent 0 through 127. Six unsigned bits represent 0 through 63. Sixteen signed bits represent -32,768 through 32,767. Truncating, adding or shifting a number can change its meaning if the result width is too small or an operand unexpectedly becomes unsigned.

Use explicit widths where the design depends on a range: `3'd2` for a state, `16'sd256` for a signed Q8 unit, and a wider sum when carry is possible. A concatenation such as `{1'b0,value}` is unsigned unless deliberately cast; `$signed(...)` appears around appropriate physics operands for this reason.

The debouncer illustrates why parameterization must include width. Its count has to reach `STABLE_CYCLES-1`. With `STABLE_CYCLES=1048577` (1,048,577 samples), the terminal value is 1,048,576. A fixed 20-bit counter cannot reach it, so the long-interval version never accepted a stable press. The final code uses `$clog2(STABLE_CYCLES)`, with a minimum of one bit, to size the counter for the positive configured interval.

Custom source files use `default_nettype none` and explicit net ports. This turns a misspelled signal into a compilation failure instead of silently creating a one-bit wire. The directive is restored to `wire` at each file's end so it does not impose new requirements on the supplied modules. It is a maintenance safeguard; it does not improve gameplay by itself.

5. Architecture, ownership and the module connections



Renderers consume state. They never update the physical world or score.

Hardware architecture and signal flow

The diagram emphasizes data ownership. Rules own score, lives and game state. Physics owns physical ball state and flipper angles. Collision logic computes a correction from an isolated snapshot. Renderers consume display copies; they never change a ball's physical position. This prevents a visual effect from accidentally becoming a physical obstacle.

File or module	Main responsibility	Important inputs	Important outputs
`My_Top_Module`	Clocks, resets, input routing, module wiring, scene capture	Clock, buttons, switches, mouse interface	JC, VGA, LEDs, seven-segment pins

File or module	Main responsibility	Important inputs	Important outputs
`pinball_input`	Synchronize and debounce board buttons	Raw buttons, clock, reset	Held `down`, pulse `pressed`
`pinball_game`	States, charge, score, lives, missions, saves	Actions, `tick`, hits, final drain, lane status	State, `prepare`, `launch`, charge, HUD fields
`pinball_physics`	Q8 motion and one/two-ball scheduling	Commands, held flippers, difficulty, `tick`	Positions, angles, hits, visibility, final drain
`pinball_collision`	Combinational contact and response	Stable old/proposed positions and geometry	Corrected position/velocity, raw contacts
`pinball_effects`	Trails and two retained spark bursts	Actual hit position, selected trail ball, active/tick	Trail and particle coordinate/life arrays
`pinball_scene`	Unpack OLED's frame bundle	318-bit scene, pixel coordinates	RGB565 OLED pixel
`pinball_renderer`	Compact OLED geometry and UI	Frame state and logical pixel	RGB565 pixel
`pinball_vga_scene`	Unpack VGA's frame bundle	611-bit scene, native coordinates	RGB565 native pixel
`pinball_vga_renderer`	Native detail, HUD, font, effects	Frame state and native pixel	RGB565 pixel
`pinball_vga_clock`	Generate buffered 40 MHz	Main clock, reset	Pixel clock, MMCM lock
`pinball_vga`	Raster counters and RGB/sync output	Pixel clock, computed pixel	Pixel coordinates, RGB444, H/V sync
`pinball_sevenseg`	Active-low digit multiplexing	BCD score, main clock	`seg`, `an`, decimal point
`Oled_Display`	Supplied OLED startup and SPI	6.25 MHz clock, pixel data, reset	Pixel requests and OLED pins
`MouseCtl` / `Ps2Interface`	Supplied mouse protocol stack	100 MHz, PS/2 pins, initial settings	Position, buttons, complete-packet event

`pinball_scene` and `pinball_vga_scene` are in `My_Top_Module.v`. The entity `MouseCtl` is in `Mouse_Control.vhd`. File names and instantiated module/entity names need not be identical; named ports are the integration contract.

The central command and event contracts

Signal	Meaning and timing contract
`tick`	One main-clock-cycle enable roughly every 4.167 ms; not a separate physics clock
`prepare`	One-cycle request to put selected balls in the shooter lane and cancel pending physics/effects
`launch`	One-cycle request to launch prepared balls using the stored `charge`
`active`	Rules are in PLAY; permits accepting a new physics update
`hit_events[13:0]`	A one-cycle scoring contact transaction after filtering/rearming
`hit_ball`	Identifies which ball produced the corresponding hit transaction
`drained`	Final live-ball drain; a partial two-ball drain does not spend a life
`balls_visible[1:0]`	Independent white/gold live/queued visibility bits
`in_lane`	True only when every live ball is still in its own launcher lane
`frame_begin`	Display-specific moment to capture a whole scene

Follow a normal launch. The game is READY and observes a held launch control. Every fourth `tick` changes `charge`. When the control is released, rules assert `launch`, enter PLAY, and establish eligible launch protection. On the next main-clock edge physics consumes the command and assigns launch velocities. At a later physics tick it snapshots operands, computes contacts, and commits corrected state after its isolated calculation window. Hits then become score transactions and effects. A display next captures a producer bundle at its frame boundary and paints that state for the full frame.

Every arrow in this description has a contract. If a new module expects a held level but you send a one-cycle pulse, it may operate for just 10 ns. If it expects a pulse but you send a held level, it may repeat hundreds of millions of times. Write down the meaning, clock domain, units, validity and lifetime of each signal before wiring modules together.

6. Clocks, resets, physical inputs and the mouse

Three real clocks and one slow enable

The board supplies 100 MHz. The top's four-bit divider has a bit that toggles at 100 MHz / 16 = 6.25 MHz; a BUFG routes that OLED clock globally. An MMCM multiplies the main clock by 10 to a 1 GHz VCO, then divides by 25 to produce 40 MHz for VGA. Dedicated clock resources distribute actual clocks with controlled skew.

The physics counter reaches `PHYSICS_TICK_LAST = 416665` and resets. Counting from zero includes 416,666 main-clock edges per pulse. Thus the actual rate is `100000000 / 416666 = 240.000384... Hz`, conventionally described as 240 Hz. Most game counters act on `tick` while their registers remain clocked by 100 MHz.

Teaching example:

```
// Preferred pattern for a slower operation in the existing clock domain.
always @(posedge clk) begin
    if (reset)
        timer <= 0;
    else if (tick && timer != 0)
        timer <= timer - 1'b1;
end
```

Using `always @(posedge tick)` would turn an enable pulse into another clock domain. It would require different clock routing, reset treatment and timing constraints. The clock-enable pattern keeps short-lived actions, rule pipelines and the collision scheduler in one main domain.

Reset is part of the design, not just a testbench convenience

The top initializes a 32-bit all-ones shift register and shifts zeros through it after configuration. Its nonzero value keeps system reset asserted at startup. Switches pass through two synchronizer registers; synchronized SW15 also asserts reset. Reset clears score and high score, menu selection, commands, physics, effects and mouse configuration.

The VGA clock generator has a lock indication. If lock disappears, the VGA reset pipeline asserts asynchronously; after lock, the pipeline releases reset through pixel-clock edges. This allows video logic to restart coherently even when the pixel clock was temporarily unavailable. Do not treat an unlocked generated clock as a valid display clock.

Synchronization, debounce and edge detection are different

A physical button can change near a clock edge and can bounce mechanically. Two synchronization registers reduce the chance that metastability propagates into game logic. They do not by themselves eliminate bounce. Each input's debounce count advances only while the synchronized sample disagrees with the accepted level. When the count reaches its configured terminal value, the accepted level changes once.

At 100 MHz, 500,000 samples correspond to 5 ms. The five channels have independent counters, so bounce on one button cannot delay another. `down` is a held level; `pressed` is a one-cycle rising event. Left/right flippers use held controls, while menu toggles and pause use events. A held U therefore toggles the ball mode once, not continuously.

The mouse is a supplied interface plus a game adapter

The Basys 3's USB-A host connection is bridged to PS/2-style signals for the FPGA. The protected `MouseCtl` and `Ps2Interface` handle protocol communication. The game supplies four initial configuration requests: maximum X 95, maximum Y 63, initial X 48 and initial Y 32. They use logical OLED coordinates, so one control layout can serve both displays.

Mouse buttons and coordinates can update during different parts of a packet. The top delays `new_event` and stores previous complete-packet buttons and cursor coordinates. Held flipper/launch controls use those committed button levels. On the dispatch cycle, click hit-testing uses the final packet coordinates, avoiding a click based on the previous Y position.

Click detection compares the current complete-packet left button with the previous committed left button. SW0 gates mouse actions. The clickable rectangles are deliberately bounded: mode, difficulty, Start, Resume and Restart. A decorative title or empty margin is not an action. The mouse cursor and hover graphics use the same logical layout; different-looking buttons must still have matching control regions.

Board and mouse controls are combined with OR conditions for held actions. Changing input devices therefore does not require another physics engine. Nudge is a separate edge action, rejected in the shooter lane and limited by a 480-tick cooldown. A similar adapter pattern works for a joystick, keypad or another peripheral: translate its protocol into the game's existing held and event signals.

7. From controls to game rules

The module `pinball_game` answers questions such as "is the game running?", "does this drain cost a life?", and "what score should this contact earn?". It does not calculate ball trajectories or produce display pixels. That separation is the first design idea to keep when rebuilding the project: the rules consume events from the physics datapath and publish state for the displays. `My_Top_Module` connects these modules and translates physical inputs into commands.

Every game register uses the 100 MHz `clk`. The slower `tick` is a one-clock enable, not a separate game clock. `PHYSICS_TICK_LAST=416665` makes one tick every 416,666 system clocks, approximately 240.000384 Hz. Calling it 240

Hz is appropriate for human-scale timing. The rules change state on any 100 MHz edge, but most timers change only when `tick` is true. A 480-tick countdown therefore takes approximately two seconds, rather than 4.8 microseconds.

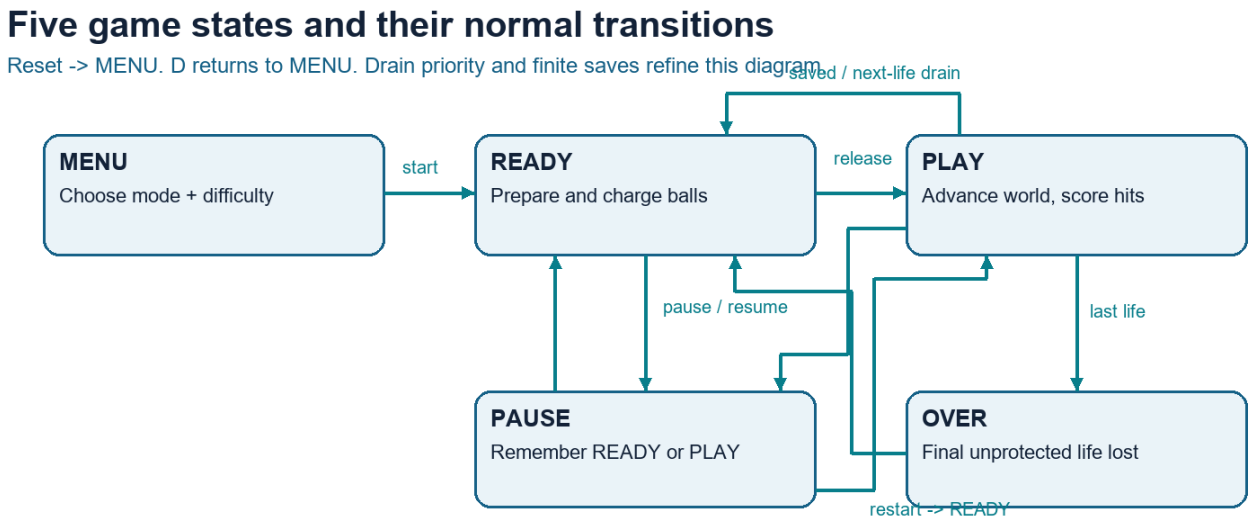
There are two input meanings. A held level remains true throughout a press; a pulse requests one action. Debounced `buttons` are held levels and `button_press` contains rising-edge pulses. Confusing them would make a held menu button cycle difficulty thousands of times or would charge the spring for only 10 ns.

Command in the RTL	Source and meaning
`start_press`	C rising edge or a mouse click in the displayed start/restart/resume region
`launch_down`	Held C or held mouse middle button
`pause_press`	U rising edge; its meaning in MENU is handled separately
`menu_press`	D rising edge
`level_left`, `level_right`	L/R rising edge or a click on the appropriate half of the menu difficulty row
`left_pressed`, `right_pressed`	Held L/R or mouse left/right button; routed directly to physics
`nudge_request`	C rising edge or middle-button rising edge during PLAY, outside the lane, with the cooldown expired

SW0 enables mouse commands; SW15 resets the system. SW1 has no mode-selection role. The mouse held controls use `mouse_prev`, the last complete packet, so partially received button bytes cannot release a spring or move a flipper prematurely. A click additionally uses `mouse_event_d`, ensuring its cursor coordinates belong to that completed packet.

In MENU, U or the mouse ball-mode row toggles `menu_two_ball_mode`. Starting a game copies the selection into `two_ball_mode`. The connection `menu_two_ball_mode ^ mode_change` deliberately handles U and C arriving together: the new game uses the selection the player just requested. The chosen mode remains fixed for that game. `nudge_cooldown` loads 480 ticks and counts down only during active play; the top level also prevents a nudge while all surviving balls are in the shooter lane.

8. The five-state controller and event priorities



Five game states

A finite-state machine stores the current operating mode and selects which transitions are legal. Here `state` is a three-bit register with five named values. The names are useful because readers can reason about READY without memorising the binary encoding.

State	Value	Main behaviour and exits
`MENU`	0	Choose difficulty and ball mode; start prepares a new game
`READY`	1	Hold/release the launcher; pause can suspend preparation
`PLAY`	2	Physics active, scores accepted, play timers advance
`PAUSE`	3	Freeze ongoing play; U or a start/resume command restores `resume_state`
`OVER`	4	No lives remain; start prepares a fresh game

`active` is the combinational expression `state == PLAY`. Physics uses this enable to decide whether it may begin another update. `prepare` and `launch` are one-clock command pulses: the clocked block clears them by default, then asserts the appropriate one when a transition needs it. Preparing resets ball placement; launching consumes the current `charge`. An illegal state falls back to MENU, while OVER stays OVER until a higher-priority command changes it.

The explicit transition order is reset first, then `menu_press`, then `new_game`, then an accepted `drained` event, then ordinary state-specific commands. This order is part of the specification. If D, U and drain arrive together, returning to MENU wins. If the last unprotected life drains on the same edge as U, OVER wins. For a non-final or saved drain, `next_life_paused` preserves the simultaneous pause/resume request and records READY as the state to resume into.

Pause needs special care because physics takes several clocks to finish an update. Setting `active=0` prevents another update but cannot erase one already calculating. `accepts_play_event` therefore remains true in PAUSE when `resume_state==PLAY`. That final hit or drain is still processed, including its save entitlement and award. A pause from READY does not accept gameplay results. The score pipeline also finishes already accepted transactions during pause or drain. This distinction between stopping new work and finishing outstanding work is useful in any FSM controlling a multicycle datapath.

Counters are not all governed by the same condition. `anim`, hit flashes and `feature_timer` advance on `tick` in every state except PAUSE. Combo, gold-target and spinner timers advance only in PLAY. The pseudorandom `rng` continues on system clocks even while paused. Thus "paused" means frozen gameplay, not that every flip-flop in the device stops changing.

9. Charge, lives and protected relaunches

`charge` is a six-bit value, 0 through 63. In READY, holding `launch_down` sets `charging`. A two-bit `charge_div` permits one power step per four ticks: approximately 60 steps per second. `charge_falling` selects whether the next step increases or decreases power. The sequence is `0,1,...,62,63,62,...,1,0,1,...`; direction reverses on arrival, with no extra endpoint hold. Each half-cycle takes approximately 1.05 seconds. Releasing after a charge asserts `launch`, changes the state to PLAY, and clamps a power below 16 up to 16 so a brief tap can leave the lane.

The first C press starts the game; continuing to hold C in READY begins charging. A READY pause freezes the current spring level, but resuming clears `charging`, `charge`, `charge_div` and direction. This prevents a release during pause from unexpectedly launching old stored power. The player can charge again after resuming.

`lives` counts remaining turns, including the current one. A new game sets it to three. An unprotected drain changes three to two, then two to one; draining with one remaining sets zero and OVER. A "turn" can contain a pair of balls. `pinball_physics` reports `drained` to the game only when the final surviving partner drains, so losing the first ball in two-ball mode does not deduct a life or prepare another shot. Both balls are prepared and launched together for the next turn.

Protection deliberately allows more than three shots. On a fresh-life launch, `launch_save_eligible` loads `save_timer=720`, approximately three seconds. The timer decreases only during PLAY and when `in_lane` is false. In two-ball mode, `in_lane` remains true only while every surviving ball is in its lane, so the countdown begins when either survivor enters the table. Completing both side targets grants `save_timer=1440`, approximately six seconds.

If the final drain occurs while `save_timer!=0`, or on the same edge as `target_complete`, the game prepares another shot without reducing lives. It consumes protection by clearing the timer and setting `launch_save_eligible=0`. That saved relaunch receives no automatic three-second save. A later unprotected life loss makes the next life eligible again. The protection is shared by the turn; it is not a separate timer for each ball. The renderers use the same published `save_timer` to make protected balls and their trail bright blue.

10. Contact masks, combos and collection rewards

`hit_events` is a 14-bit event mask. Each bit identifies a scoring contact, not a persistent overlap. Physics filters raw collision contacts with per-ball arming masks, so one ball resting near a bumper cannot earn points every tick. In two-ball mode the two results are emitted as separate transactions, allowing both contacts to score.

Bits	Contact group	Base units per contact	Collection memory
0-2	Left moving, right moving, and central bumpers	1	`mission_progress[2:0]`
3-4	Left/right side targets	2	`target_progress[1:0]`
5-7	Three upper rollover lanes	1	`table_progress[2:0]`
8	Spinner	1	No collection bit
9-10	Left/right slings	1	No collection bit
11-13	Three upper stand-up targets	2	`table_progress[5:3]`

One unit is half a displayed point. A transaction ORs its bits into the remembered collection, for example `mission_next = mission_progress | hit_events[2:0]`. AND-reduction `&mission_next` means all three bumper bits are set. Completion awards its bonus and resets just that collection, making it repeatable. The bumper mission, side pair, rollover trio and stand-up bank award respectively 10, 5, 6 and 8 half-point units: 5, 2.5, 3 and 4 points.

Combo is a multiplier for the base contact units. With `combo_timer==0`, the next transaction uses ×1. Each following transaction within 480 ticks increases it through ×2 and ×3 to a maximum ×4, and reloads the two-second window. Multiple bits in one transaction increase combo once; they all use that transaction's multiplier. Flat collection and gold bonuses are added afterwards, so they are not multiplied.

The gold target is one of bumper bits 0-2. `bonus_timer` starts at 720; the target is active for timer values 480 down to 1, giving about one second waiting followed by two seconds active. A gold hit grants 5 units and restarts the cycle. Expiry also restarts it. The LFSR `rng` chooses a target from its lowest two bits, mapping value 3 back to target 0. This is inexpensive variation with a slight selection bias, not a uniform random-number generator.

All earned bonuses accumulate even if several complete together. Only the displayed `feature_event` has priority: bumper jackpot, side reward, table completion, gold, then combo. Event codes are 1 gold, 2 combo, 3 bumper jackpot, 4 saved ball, 5 side reward, 6 nudge and 7 table bonus. Notifications load `feature_timer=240`, about one second; a newer qualifying notification replaces the previous one. Each hit also loads its `flash_timer` to 24 ticks, about 100 ms. Spinner hits add three to `spinner_phase` and load 120 ticks; another phase step occurs every four active ticks while that timer runs.

11. Half-point scoring and the three-edge pipeline

Four seven-segment digits can represent 0000-9999. `score_bcd` and `high_bcd` therefore hold four binary-coded decimal nibbles: `16'h0123` represents decimal 123. It is not the binary integer `16'd123`. The carry function `add_bcd` adds each digit and propagates a decimal carry; a carry beyond the thousands digit saturates the result at `16'h9999`. Valid BCD values also compare in numeric order, so the independent `score_bcd > high_bcd` comparison is safe.

Scoring is pipelined to avoid placing combo arithmetic, conversion and four-digit carry on one 100 MHz path. Stage 1 accepts the event and registers `award_units_1`. Stage 2 adds the stored `score_half`, converts the whole part to `award_bcd_2`, and preserves the low bit as the new fractional remainder. Stage 3 commits the BCD addition and updates `last_award_bcd`. Valid flags `award_valid_1` and `award_valid_2` distinguish a real zero-point fractional credit from no transaction.

Consider starting with score and high score both 0098, half remainder 0, and expired combo. Two consecutive bumper-contact transactions earn 1 unit at ×1 and 2 units at ×2. No collection or gold bonus applies:

Rising edge	Newly completed action
E0	Capture A = 1 half-point unit
E1	Convert A: whole credit 0, remainder 1; capture B = 2 units
E2	Commit A: score stays 0098; convert B + remainder: credit 1, remainder 1
E3	Commit B: score becomes 0099; `last_award_bcd=0001`
E4	Independent high-score register catches up to 0099

The earned total is 1.5 points, with 0.5 retained for a future transaction. Conversion happens in transaction order, so adjacent awards cannot lose that remainder. The largest synthetic transaction, all 14 contacts at ×4 plus every applicable flat bonus, is 110 units or 55 points. That bound justifies seven-bit award registers and a small compare/subtract decimal converter instead of a general divider. At saturation, `last_award_bcd` still describes the transaction's whole credit rather than the smaller increase visible in the capped score.

Reset clears both scores. A new game clears the current score, fractional carry and pipeline valid bits but preserves the session high score. Returning to MENU also preserves the high score. A drain resets combo but preserves accumulated score and collection progress; those collections reset on completion or a new game.

12. Sequential reasoning and a rebuilding exercise

In `always @(posedge clk)`, nonblocking assignments read the pre-edge values and schedule new register values. This explains why E1 uses A while also accepting B, and why high score trails a committed score by one clock. It also explains timer tests such as `if(combo_timer==1)`: the condition sees the old one while decrement schedules zero. Later assignments to the same register in this block win, intentionally allowing a fresh hit to reload a timer after the generic countdown would decrement it.

For a small rebuild, first implement MENU, READY, PLAY and OVER with one ball, a held spring control and one bumper event. Add PAUSE and a remembered resume state next. Only after those transitions work should you add a decimal score and staged awards. This progression exercises the EE2026 concepts of combinational next-value logic, sequential registers, FSM control, counter enables, bit masks, numeric encoding and controller/datapath interfaces.

An illustrative pattern, not a drop-in replacement, is:

```
always @(posedge clk) begin
    launch <= 1'b0;
    if (reset) state <= MENU;
    else if (state == READY && released_after_charge) begin
        launch <= 1'b1;
        state <= PLAY;
    end
end
```

Common mistakes are treating a held button as an event; using a tick as an unconstrained clock; giving a 1440-tick timer only ten bits; adding BCD as binary; discarding in-flight scores on pause; or renewing launch protection after every saved shot. Specify simultaneous-command priorities before coding rather than relying on accidental source order.

Check your implementation by answering these questions: What happens when pause and final drain share an edge? Does a two-ball first drain deduct a life? Can three ×1 half-point contacts eventually show one point with a remainder? Does MENU/restart erase pending credits? Does a saved relaunch get protection again? Are counters frozen by the intended enable, and can each register represent its maximum load? The supplied `tb_game` and `tb_two_ball_game` demonstrate these boundary cases explicitly.

13. Motion is a sampled calculation, not a moving pixel

Read `pinball_physics.v` for the stored world state and scheduling, then `pinball_collision.v` for the combinational contact calculation. The physics module stores position, velocity, shooter-lane status and visibility separately for each ball. It also owns the moving bumper centres and flipper angles. The collision module receives a snapshot of these quantities and returns corrected position, corrected velocity and a 14-bit contact mask. Scoring belongs to `pinball_game.v`; collision detection does not directly add points.

The board clock is 100 MHz, but the world advances on a 240 Hz enable named `tick`. One world step therefore represents approximately 4.167 ms. Registers remain clocked at 100 MHz; the enable decides when a new physical step may begin. This distinction connects directly to synchronous digital design: a slowly changing game can use a fast clock without inventing another clock domain.

The logical table uses OLED-sized coordinates: x increases to the right and y increases downwards. Consequently positive vertical velocity means falling, negative vertical velocity means rising, and gravity is positive. A ball is more than its displayed integer coordinate. Between x=20 and x=21 there are 256 representable fractional positions.

Signed fixed-point arithmetic and its units

The position and velocity registers are `signed [15:0]`, with eight fractional bits. The project calls this Q8: divide a stored integer by 256 to recover a logical pixel quantity. Be careful because other references use Q-format names differently; the actual bit width and scale define the representation here.

Quantity	Stored value example	Meaning
Position	10240	40 logical pixels
Position	10368	40.5 logical pixels
Velocity	128	0.5 logical pixels per physics step
Velocity	-256	-1 logical pixel per physics step
Gravity	2	2/256 logical pixels per step squared

A signed 16-bit register ranges from -32768 to 32767, corresponding to -128 through approximately 127.996 pixels. This covers the 96×64 table, the drain at y=64, and temporary negative values without needing floating-point hardware. Signed arithmetic is essential: interpreting -256 as unsigned gives 65280, an enormous and incorrect positive speed.

For a normal-difficulty free-flight example, suppose:

```
position_y = 40*256 = 10240
velocity_y = 64       = 0.25 pixels/step
gravity    = 2        = 0.0078125 pixels/step^2

accelerated_vy = 64 + 2 = 66
proposed_y     = 10240 + 66 = 10306 = 40.2578125 pixels
```

If no collision occurs, these become the new registers. The next step adds gravity again: velocity becomes 68 and y becomes 10374, or 40.5234375 pixels. This is semi-implicit Euler integration: update velocity first, then position using the updated velocity. At 240 steps/s, stored velocity 64 means 60 pixels/s, while normal gravity means `2*240*240/256 = 450 pixels/s^2`. These are table coordinates, not physical metres.

The OLED receives the integer part after clipping to x=0..95 and y=0..63. The native VGA renderer also receives fractional positions. Its table mapping uses eight screen pixels per logical pixel, with offsets x=16 and y=32. Thus x=40.5 can appear at native x=340 instead of being rounded to the same position as x=40. The physics is shared by both outputs; display resolution does not change game speed.

Limits and why the ball remains playable

`clamp_horizontal` and `clamp_vertical` deliberately cap velocity. These are experience decisions as well as numerical bounds:

Limit	Q8 units per step	Logical pixels/s
Horizontal, either direction	256	240
Upwards	320	300
Downwards before the lower region, or in the shooter lane	160	150
Downwards outside the lane at y >= 38	76	71.25

Difficulty chooses gravity 1, 2 or 3, giving 225, 450 or 675 pixels/s². It does not replace these speed caps. From y=38 to the flipper pivot height y=53, the capped descent takes at least about `15/71.25 = 0.211 s` if unobstructed. This leaves a useful reaction interval on a small table. Extremely fast, physically realistic motion would often be less enjoyable here.

Preparation places ball 1 at (89,56) and optional ball 2 at (89,60). A single launch sets each available ball's initial vertical velocity to `-288-launch_power`, then subsequent physics steps apply gravity and the upward cap. Charge therefore influences a bounded launch; it does not allow unlimited speed. At the lane exit, when the proposed y reaches 15 or less, ball 1 receives horizontal velocity `-160-shot_power`, ball 2 `-128-shot_power`, and both receive vertical velocity -56. Different starting heights and exit velocities separate their paths after the shared shot.

The collision block confines an in-lane ball centre to x=85..93. Outside the lane, the left boundary is x=3; the right lane divider limits x to 83 at y>=18, with x=93 allowed in the upper opening. The top boundary is y=10. A ball drains when collision correction reaches y>=64 inside x=30..56, outside the lane. Display clipping at y=63 is presentation only: it does not stop the physical ball reaching the drain.

14. Contacts: approximating useful geometry in hardware

`pinball_collision.v` is an `always @*` combinational block. It initializes every output and temporary, then applies contact tests in a stable priority order. A `solid_contact` flag allows one main solid-object response per step: bumpers, stand-up targets, slings, side targets, flippers and guides are considered in order. Outer boundary correction still follows. Sensor regions such as rollovers and the spinner report passage without stopping the ball.

This ordered approximation is intentional. The project is an arcade model, not a general rigid-body solver. It has no ball-to-ball collision, spin, continuous contact manifold or general search for the earliest collision among all objects. Two balls may pass through each other. This keeps the hardware cost and behavior predictable.

Inflated objects and simple distance tests

Rather than calculate a circle intersecting another shape, enlarge the object by the ball radius and treat the ball centre as a point. The drawn bumper radius is three logical pixels and the ball radius is one. Its clearance envelope is an octagon described by:

```
abs(dx) <= 1024              // 4 pixels
abs(dy) <= 1024              // 4 pixels
abs(dx) + abs(dy) <= 1472    // 5.75 pixels
```

Here `dx` and `dy` are ball position minus bumper centre in Q8 units. These comparisons use absolute values, adders and comparators instead of square roots. The response chooses an axis or 45-degree normal, providing eight useful rebound directions without trigonometric hardware.

The three permanent stand-ups at (34,20), (42,20) and (50,20) use a square clearance of ±2 pixels around their centres. Slings at (17,41) and (69,41) use a diamond: `abs(dx)+abs(dy)<=1024`, a Manhattan radius of four pixels. These shapes match the inexpensive drawn forms well enough for play.

Why checking the final position is insufficient

A ball can graze a corner between two samples even though both endpoints are outside the clearance shape. This is called tunnelling. The code therefore also tests the segment from the previous ball position to its proposed position.

`swept_octagon` considers the four pairs of octagon faces. Each face has a projection coordinate perpendicular to it and a tangent coordinate along it. For a vertical face these are x and y; for a diagonal face they are x+y and x-y. Reflecting negative projection values lets one formula describe either sign of a face.

Suppose the old projection P0 is outside boundary B, and the new projection P1 crosses inward. The crossing fraction would be:

```
D = P0 - P1        // positive denominator
N = P0 - B
t = N/D
crossing_tangent = T0 + (T1-T0)*N/D
```

Crossing the infinite line is insufficient: the crossing must fall on the finite face. If its tangent extent is ±L, division can be removed:

```
I = T0*D + (T1-T0)*N
-L*D <= I <= L*D
```

This is what the signed wide `intercept` and `tangent_bound` calculations implement. For example, with B=1024, L=448, P0=1100, P1=900, T0=300 and T1=500, D=200, N=76 and I=75200. Because L*D=89600, the crossing lies on the face: its mathematical tangent is 376. With T0=500 and T1=700 instead, the crossing tangent becomes 576, beyond 448, and the segment is a near miss. A collision detector that only asks whether the line was crossed would falsely bounce the second trajectory.

Stand-ups reuse `swept_clearance` for rectangular clearance. The bounded step sizes allow it to select the old outside face instead of building every face calculation separately. The sling diamond becomes a rectangle in transformed coordinates `u=x+y`, `v=x-y`, so the same function handles its swept corners. The 17-, 18- and 36-bit intermediates matter: products can overflow even when each coordinate fits in 16 bits. Width selection is a direct application of finite-width arithmetic taught in digital design.

Correct penetration, reflect only inward motion

Detection is followed by two separate actions: move an overlapping ball outside the object, and change velocity only if it approaches the surface. Otherwise an already departing ball can be reflected back into the object every step and appear stuck.

For a right-facing bumper surface, positive x is outward. A relative x velocity below zero approaches the surface, so it reflects; positive x is already leaving. For a diagonal, the approach expression adds the signed components along the selected normal. The `approach<0` test also gates the hit event. Axis bumper rebounds retain roughly 7/8 of incoming speed, add a small impulse of 24 Q8 units, and enforce a useful minimum of 160. Diagonal rebounds swap/sign-change the two components, then add a 32-unit outward impulse. These impulses deliberately make a pinball bumper energized rather than a passive wall.

Moving bumpers require relative velocity. If the bumper itself has velocity c, reflect `ball_velocity-c`, then add c back. Old and new relative positions also subtract the matching old and new bumper centres. Mixing an old ball position with a new bumper centre would distort the swept segment and create false hits.

The left bumper moves vertically at x=26, y=22..28; the right moves horizontally at y=22, x=58..66. Each accepted world step moves a centre by four Q8 units, or `4*240/256 = 3.75 pixels/s`. End comparisons reverse direction. Both ball calculations see one shared old/next pair, and the centres commit once after the complete world step. Do not advance the world once per ball: that would make two-ball mode move bumpers twice as fast.

Flippers are controlled surfaces with different responses

Each flipper has angle states 0..4. Holding the button advances one angle per accepted physics tick; releasing retreats one. Four transitions take about 16.7 ms. The left pivot is (21,53), the right (65,53), and each reaches 18 pixels toward the middle. A lookup function gives inner-tip offsets +5,+3,+1,-2,-4, approximating rotation without sine/cosine.

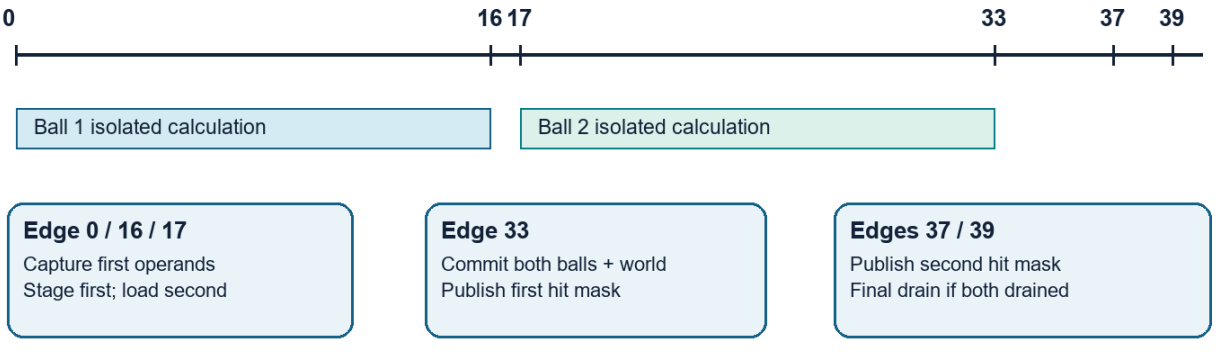
The surface height is `53*256 + local_x*tip_offset/18`. The previous ball height is checked as well as the proposed height. A ball travelling below a flipper should continue below it, rather than being teleported onto its top. A rising flipper may sweep into a nearby ball within its clearance band.

A rising flipper gives a strong upward and inward impulse that grows toward the tip. A held raised flipper gives a weaker return. A released flipper halves sufficiently fast downward speed, while a low-energy ball rolls toward its tip instead of receiving an endless upward impulse. Post-collision speed limits still apply. Side guides join the visible pivots; matching rendered geometry and collision endpoints prevents invisible rails.

15. Sharing one collision engine between two balls

One world step: shared calculation, atomic results

Each numbered edge is a 100 MHz clock edge (10 ns). Tick period is about 4.167 ms.



Consumers see registered hit/drain pulses on the following system edge.

Shared two-ball calculation timeline

The expensive combinational collision block exists once. Physics chooses a ball's operands, snapshots them into `collision_*` registers, waits for the calculation, and stores its result. This is resource sharing controlled by a small sequential scheduler, not two concurrent software function calls.

At 100 MHz one clock is 10 ns. A collision transaction gets 16 clocks, or 160 ns. One-ball mode commits after that window. With both balls launched, the first result goes into `stage_*` registers while the displayed/live ball positions remain unchanged. A separate handover clock selects and captures ball 2's operands, followed by its 16-clock window:

```
edge 0     capture ball 1 and the world's old/next bumper centres
edge 16    stage ball 1 result; select second-ball work
edge 17    capture ball 2 operands
edge 33    commit both ball results and the moving centres together
```

The pair costs 330 ns, negligible against the 4.167 ms physics period. Atomic final position commits prevent a renderer or effects module observing a newly moved ball 1 alongside an old ball 2. If only one survives, it returns to the 16-clock path; losing one ball does not immediately end the round.

Both balls can hit the same object in one step. OR-ing their contact masks would count that as only one contact. At the pair commit, the first mask appears with `hit_ball=0`; the second appears four system clocks later with `hit_ball=1`. If both balls drained, the final `drained` pulse appears six clocks after commit. Registered outputs are consumed by the game module on the following clock edge, so the relative sequence remains first contact, second contact, final drain. This preserves both awards before the game leaves PLAY. The four-clock spacing also leaves time for combo/progress changes from the first contact to influence the second transaction.

Each ball has separate `hit_armed` bits and 14 cooldown counters. A scored contact disarms its bit and starts a 16-step cooldown, about 66.7 ms. Time expiry alone does not rearm it: the ball must also leave a larger object-specific envelope. This hysteresis prevents repeated points while resting against a bumper or sitting inside a sensor. The other ball retains independent eligibility.

A one-clock nudge may arrive between physics ticks, so `nudge_pending` holds it. The accepted batch captures it once for both calculations; a late nudge waits for the next batch. Each eligible ball receives ±48 horizontally toward the middle, and -80 vertically, subject to limits. In-lane balls receive no nudge. This avoids the bug where a late control pulse affects only ball 2.

Pause prevents accepting another world step. An already accepted transaction finishes normally, including deferred contact/drain outputs; the rules module explicitly accepts that final event while paused. Reset or `prepare` instead cancels the calculation, staging and pending event timers. Preparation is used for a new round/menu, so stale contacts cannot arrive in the newly prepared game.

The timing constraint is a promise made by the RTL

`Pinball_Physics_Timing.xdc` applies a 16-cycle setup exception and a matching 15-cycle hold exception from isolated collision snapshots to specified result-state registers. Ordinary control paths still have a 10 ns requirement. The calculation is not required to settle in one clock because its operands remain unchanged and its result is not captured until 16 clocks later.

This is legal only because the scheduler actually enforces that contract. Declaring a slow path multicycle does not make unsafe hardware correct. Changing snapshots midway, committing early, or broadly applying the exception to controls would invalidate it. The paired hold adjustment prevents the setup exception from also imposing an unintended later hold requirement. The build checks that the expected snapshot registers exist; timing reports must still be inspected after implementation.

16. Recreate the physics incrementally

Begin with one signed Q8 position/velocity pair and a tick enable. Simulate free flight against the worked example before adding graphics. Add speed clamps and axis walls, separating penetration correction from inward-only reflection. Then implement one stationary bumper and test it from every side. Add flippers and a drain; ensure a released flipper cannot sustain the ball forever. Only after this works add sensors, scoring pulses, moving-object relative coordinates and swept corners. Finally share the engine with a second set of registers and add atomic commit/event scheduling. This order makes each failure attributable to a small new idea.

Useful exercises have measurable outcomes:

- Send a ball away from an overlapping wall: its position corrects, but velocity does not reverse inward.
- Compare one exact corner crossing with a near miss: only the crossing reports a contact.
- Keep a ball inside a sensor for many ticks: it scores once; exiting and re-entering after cooldown permits another score.
- Hit one bumper simultaneously with two balls: two masks appear four clocks apart, with independent arming bits.
- Pause after accepting a batch: that batch finishes once; later ticks cause no movement.
- Prepare during the second calculation: no staged position, contact or drain leaks into the next round.
- Drain one ball: the other continues. Drain the last: exactly one round-drain pulse follows the final contacts.

`tb_physics.sv`, `tb_collision.sv`, `tb_multiball.sv` and `tb_two_ball_game.sv` cover these kinds of contracts. The additional archived geometry oracle under `build/final_physics_audit/` compares swept cases against an independent mathematical model. This is more informative than checking that a simulation simply runs: state the numerical result, event count and latency that correct hardware must produce.

17. How one game produces two different pictures

The display system answers a very specific question: **given the current scene and the coordinate of the pixel being scanned, what colour should that pixel be?** It does not advance physics or award points. That separation lets one authoritative game feed a small OLED and a larger VGA screen without duplicating the game rules.

The main files divide this work as follows:

File or module	Responsibility
`pinball_renderer.v`	Compact 96×64 picture, 3×5 characters, OLED-sized shapes and trail dots
`pinball_vga_renderer.v`	Native 800×600 picture, 5×7 characters, fine movement, larger HUD and effects
`pinball_effects.v`	Clocked storage and movement of trail samples and impact sparks
`pinball_vga.v`	Raster counters, blanking, horizontal/vertical synchronization and registered VGA colour outputs
`pinball_vga_clock.v`	MMCM and clock buffers producing the 40 MHz pixel clock
`pinball_scene`, `pinball_vga_scene` in `My_Top_Module.v`	Unpack a frame's scene bundle and connect the appropriate renderer
Protected `Oled_Display.v`	OLED initialization, frame scheduling and SPI transmission

The OLED and VGA renderers are **combinational circuits**. An `always @*` block describes comparisons, arithmetic and multiplexers that settle to a colour. It is not a CPU painting the screen by executing a loop over every pixel. The display controller supplies successive coordinates; the hardware answers each coordinate separately.

18. Procedural drawing and layer priority

This project mostly draws mathematically rather than reading a full-screen bitmap. A rectangle needs bounds comparisons. A diamond can use `abs(x-cx)+abs(y-cy)<=radius`. A circle uses squared distance, with carefully limited operand widths. Rails use constant coordinates or line equations; flippers use signed distance from a short line segment. Lighting and outlines are extra tests near each shape's edge.

The following is a **simplified teaching example, not a replacement module**:

```
always @* begin
    pixel = BACKGROUND;                  // Every path has a default.
    if (inside_table) pixel = TABLE_COLOR;
    if (inside_trail) pixel = TRAIL_COLOR;
    if (inside_bumper) pixel = BUMPER_COLOR;
    if (inside_ball) pixel = BALL_COLOR;   // Latest assignment wins.
end
```

Blocking assignments express an ordered selection of the final colour. Several conditions may hold for the same pixel; the last applicable assignment wins. This is the painter's-order rule. Synthesis builds priority logic from it, so layers affect both visual correctness and timing.

The actual renderers draw subdued backgrounds and comic reactions before trails and table objects. Trails remain beneath obstacles. Text, panels and pointers come later; **live PLAY balls are drawn last**, above notifications, sparks and the pointer. READY, PAUSE and OVER have their own overlay treatment, so do not generalize the PLAY rule to every state.

Trail samples are painted from index **7 down to 0**. Index 0 is the newest. An older dim square must not overwrite a brighter fresh square where they overlap. This is a useful example of a visual bug that can exist even when every coordinate and colour individually looks correct.

Keep collision geometry and visible landmarks consistent. The bright ring around a bumper is an animation, not an enlarged collision boundary. The circuit cat is decoration, not an obstacle. When adding a wall, change and verify its intended collision representation as well as its picture; a drawn wall without a collision feels hollow, while a hidden collision feels unfair.

19. Coordinates: logical playfield versus native VGA

Physics uses the compact table's coordinate system, with eight fractional bits. For example, a raw X position of `10816` represents `10816/256=42.25` logical pixels. The OLED shows the integer part, `42`; VGA can show the fractional motion more finely.

The native table fits inside a 768×512 region with origin `(16,32)`. The conversion is:

```
native_x = 16 + floor(x_fp / 32)
native_y = 32 + floor(y_fp / 32)
```

Dividing Q8 coordinates by 32 is equivalent to multiplying the logical position by eight and discarding only fractions smaller than one native pixel. In RTL, slices such as `ball_x_fp[14:5]` implement that conversion by wiring. For `(42.25,35.5)`, raw positions are `(10816,9088)`, OLED centre is `(42,35)`, and VGA centre is `(354,316)`. One VGA pixel represents one-eighth of a logical pixel. Five low Q8 bits are discarded; three fractional bits beyond the logical integer position remain useful.

The VGA renderer receives the **full native coordinate**, not the OLED colour expanded into an 8×8 square. Its first ball is a radius-eight circle with highlights, its second ball uses a chamfered silhouette to limit additional arithmetic, and its bumpers have circular faces and smooth rings. Native text and menus have their own placement and scale. Consequently the large screen gains detail and finer motion, while the OLED remains legible at its physical resolution.

The legacy logical `x`/`y` outputs of `pinball_vga` still exist, but Top leaves them unconnected. `pixel_x` and `pixel_y` drive `pinball_vga_scene`. Connecting the legacy outputs instead would throw away the native rendering advantage.

20. Colours and small font memories

Both renderers return **RGB565**: red in bits `[15:11]`, green in `[10:5]`, and blue in `[4:0]`. There are 5, 6 and 5 bits per component. Full red is `16'hF800`, full green is `16'h07E0`, and white is `16'hFFFF`. A protected ball's principal bright blue shade, `16'h05BF`, has red 0, green 45/63 and blue 31/31, giving a cyan-blue rather than a dark pure blue.

The OLED consumes all 16 bits. Basys 3 VGA has four output bits per channel, so `pinball_vga` selects:

```
red   <= visible ? pixel[15:12] : 0;
green <= visible ? pixel[10:7]  : 0;
blue  <= visible ? pixel[4:1]   : 0;
```

These are the most significant four bits of each channel; they are not three adjacent four-bit slices of the 16-bit word. Misreading this packing causes wrong colours. In the blue example the VGA outputs are `(0,11,15)`.

The font functions are small constant lookup tables: a character code selects a 15-bit OLED glyph or a 35-bit VGA glyph. Calling them ROMs describes their behaviour, not a guarantee of block-RAM inference. With small constant tables, Vivado may implement them in LUT logic.

For the OLED, `A` is `010_101_111_101_101`: five rows, each three bits wide. After selecting a message's character and its local glyph coordinate `(gx,gy)`, bit `14-(gy*3)-gx` determines whether to draw ink. Native VGA similarly uses `34-(gy*5)-gx`. Blank glyph cells leave the previously drawn colour visible. Character spacing prevents adjacent glyphs from touching.

Scale comes from coordinate shifts. A doubled character divides local coordinates by two; a fourfold character divides by four. These operations repeat glyph pixels without storing additional fonts. Each renderer selects one text region and passes it through a shared decoder, avoiding a separate font circuit for every label. The code's signed arithmetic, explicit widths and range guards remain important: subtraction and glyph indexing must not wrap outside the selected region.

21. OLED scan and the supplied SPI driver

The OLED driver's linear `pixel_index` runs through 6144 positions. Top computes `oled_x=pixel_index%96` and `oled_y=pixel_index/96`. Index 0 means `(0,0)`, 95 means `(95,0)`, 96 means `(0,1)`, and 6143 means `(95,63)`. These are constant division and remainder operations that synthesis must implement; they are not software library calls.

Top derives the OLED clock by buffering bit 3 of a four-bit divider, giving `100 MHz/16=6.25 MHz`. The protected controller initializes the display, sets its column/row address range and serializes each RGB565 word in **16 clock cycles**. Its frame divider targets approximately 60 frames per second. The pixel payload alone occupies `6144×16/6.25 MHz≈15.729 ms`; initialization and frame commands are separate from that payload.

Read the actual edges carefully: `Oled_Display` updates its state, counters and SPI shift register on the **falling edge** of its `clk`. While busy, `sclk` follows that clock; the falling-edge update prepares data for the next rising SPI edge. Top captures `oled_scene` on the **rising edge** when `oled_frame` is asserted. This gives the renderer time to settle before the driver takes the next pixel word on a falling edge. Describing both modules as rising-edge consumers would hide an important half-cycle timing relationship.

`frame_begin` identifies a frame boundary; `sample_pixel` identifies pixel sampling opportunities; `sending_pixels` indicates the streaming phase. Top uses `frame_begin` for scene capture and supplies combinational `pixel_data` continuously. It does not need a framebuffer or to alter the supplied controller. The JC wiring is defined by Top and the constraints; `JC[2]` is tied low because that OLEDrgb pin is unconnected.

22. VGA raster timing and the pixel clock

VGA requires a continuously timed raster. The monitor receives colour values during active video and synchronization pulses during blanking. The 800×600 mode is:

Interval	Horizontal clocks	Vertical lines
Visible	800	600
Front porch	40	1
Positive sync	128	4
Back porch	88	23
Total	1056	628

Counters `h` and `v` cover `0..1055` and `0..627`. Horizontal sync is high for `840<=h<968`; vertical sync is high for `601<=v<605`. RGB is forced to zero outside `h<800 && v<600`. At 40 MHz, frame rate is `40,000,000/(1056×628)≈60.317 Hz`. Do not substitute 25 MHz and keep these counts: that changes the video mode.

The MMCM multiplies the 100 MHz input by ten to a 1 GHz internal oscillator, then divides by 25 to obtain 40 MHz. BUFG buffers distribute the output and feedback clocks. Top holds the VGA path in reset until lock is established, with reset release synchronized to the pixel clock.

Colour and sync outputs are registered together. At a rising edge they use the pre-edge `h`/`v`, while the counters advance for the next pixel. This consistent one-cycle registration is intentional; delaying only colour or only sync would misalign the picture. During blanking, native renderer coordinates are clamped to zero instead of allowing a shortened coordinate bus to wrap.

An external active VGA-to-HDMI converter translates the board's VGA output for an HDMI monitor. This project does not generate HDMI TMDS signals in FPGA logic.

23. Stable pictures: producer registers and frame snapshots

Game values change at different instants. Physics takes several 100 MHz clocks to complete a transaction, scores update on accepted events, and effects have their own storage. Sending those live wires directly to a scanning display would let a ball appear at its old position on one row and its new position on another: **tearing**.

Top first registers complete display payloads at 100 MHz in `oled_transfer` and `vga_transfer`. Each display then copies its transfer register only at its own `frame_begin`:

Bundle	Width	Contents and use
OLED scene	318 bits	State/HUD, integer ball coordinates, mode/visibility, moving centres, compact trail samples
VGA scene	611 bits	Common state/HUD plus fractional positions, native sparks and native trail samples

The scene unpackers mirror the concatenation order. The frozen `oled_scene` or `vga_scene` feeds every pixel of that frame. The two displays sample at independent boundaries, so they can show neighboring game instants; each individual picture remains coherent. A snapshot is coherent as a bundle, but it is not a promise that every producer has finished its latest game transaction at that exact instant.

The producer registers also remove coordinate-conversion logic from a tight 100-to-40 MHz crossing. Here the clocks derive from the same board oscillator, use clock buffers, and are checked as **related clocks** by timing constraints. Copying a many-bit bus at a frame edge is not a general solution for genuinely asynchronous clocks. For independent clocks, use a proper handshake, stable-data protocol or asynchronous FIFO; independently synchronizing each data bit can mix values from different updates.

24. Effects that consume little state

`pinball_effects` stores eight trail samples and two alternating banks of four sparks. Its two-bit phase divides the approximately 240 Hz physics enable by four: trails are sampled at approximately 60 Hz. New positions enter index 0; older samples shift toward index 7. Valid bits prevent uninitialized history from appearing. Renderers assign three brightness bands by sample age; these are discrete fades, not alpha blending.

In two-ball play, the shared trail follows the white ball while it survives, then the gold survivor. `trail_reset` clears old white-ball history on that handoff, and Top masks stale validity in the first outgoing scene. Otherwise an apparent line could connect two unrelated ball paths. Two balls do not each have a separate eight-sample trail.

An accepted impact supplies the corrected hitting ball's position. Four sparks replace the oldest alternating bank, allowing two closely spaced impacts to retain separate origins. Spark positions and velocities use fixed-point storage; movement occurs at physics ticks. A four-bit lifetime decreases at the slower phase, starting at 15, for roughly a quarter-second maximum burst. Out-of-screen sparks expire. Active motion freezes when paused, although an already accepted impact may finish and create a frozen burst. The OLED uses compact rings and flashes; detailed spark particles are sent to VGA.

Protection colour comes from the snapshot's `save_timer!=0`, shared by both balls and the trail. The OLED shows ceiling seconds as `S1..S6`; VGA shows `SAFE` with seconds. On expiry, normal white/gold palettes return. Renderers describe protection; the game module decides whether a drain consumes a life.

Original circuit-cat masks react to events 1/3/5/7 (major rewards), 6 (nudge) and 4 (save). They appear only while `feature_timer>120`, the first approximately half-second of a 240-tick notification. PLAY permits these reactions; READY permits the saved reaction and moves it below the charge area. Repeated combo event 2 is excluded to prevent constant interruption. OLED expresses the reaction with tiny faces, sweat or stars; VGA adds "YATTA!", "NANI!" or "PHEW!". The latest notification can replace an earlier one. It is not an event queue.

The pointer uses the same logical mouse coordinates and hover rectangles as Top's click logic. VGA transforms the pointer to the native origin/scale; OLED draws a compact cross. A hidden or disabled pointer must not highlight a button. Visible boundaries and accepted clicks should be tested together.

25. Build your own display system in manageable steps

Start with fixed colour bars and a grid. Verify RGB packing, corner coordinates, OLED order and VGA timing before connecting game state. Next draw one stationary shape, then drive it with a clocked coordinate. Add a frame snapshot and deliberately change the producer halfway through a frame to verify that the current picture stays stable. Add integer OLED motion first, then fractional native motion, HUD text, and finally effects.

For verification, check pixels at shape centres, edges and just outside bounds; inspect full-frame captures for every state. Test overlapping old/new trails, protected and unprotected balls, drained visibility, both balls over a notification, pointer corners and modal overlays. `tb_renderer`, `tb_vga_renderer`, `tb_vga`, `tb_vga_clock`, `tb_effects` and `tb_top` cover these different responsibilities. Test bundle packing with nondefault fractional values: all-zero values cannot reveal swapped fields.

Finally, rerun implementation timing. A small extra comparison late in a pixel selector can extend a priority chain enough to miss the 25 ns VGA period. Shared arithmetic, bounded widths, constant masks and late colour gates help control cost. Correct simulation proves behaviour; successful routed timing proves the implemented circuit can meet its specified clocks. Both are required for a dependable EE2026 display design.

26. Reading the source efficiently

Read interfaces before long calculation blocks. For every module, write a small card containing its clock, reset, inputs, output meanings, units and latency. For `pinball_collision`, write "pure combinational; stable snapshot in; corrected physical state out." For `pinball_game`, write "100 MHz registers; some actions each clock, most timers on tick; score pipeline can finish while paused." These cards make a large module understandable before every arithmetic detail is familiar.

Use this reading order: Top; input and clock modules; game; physics; collision; effects; scene packers; renderers; display timing and protected interfaces; constraints and tests. The order follows dependencies without demanding that you understand the expensive geometry first. In the top, search for a module name, then trace each named port back to its declaration and producer.

Question	Source location to inspect
Which button controls a feature?	Top's `buttons`, `button_press`, mouse predicates and instance ports
Who changes the game state?	`pinball_game` state transitions and command priorities
Why did score change?	Accepted hit mask, collection/combo logic and award pipeline
Why did the ball bounce?	Collision snapshots, corrected outputs and contact priority
Why did the ball stay blue?	`save_timer`, `in_lane`, pause state and display snapshot
Why is an old position still visible?	Physics commit, producer bundle and next frame boundary
Why is an object not clickable?	Logical action bounds in Top and hover bounds in both renderers
Why does simulation pass but hardware fail?	Constraints, routed timing, resets, physical interface and the programmed bitstream

Source search examples, from the project folder:

```
rg -n 'prepare|launch|drained|hit_events' FDP2026.srcs/sources_1/new
rg -n 'save_timer|save_protected' FDP2026.srcs/sources_1/new
rg -n 'collision_busy|collision_wait|load_second' FDP2026.srcs/sources_1/new
rg -n 'frame_begin|oled_transfer|vga_transfer' FDP2026.srcs/sources_1/new
```

A signal named similarly in two files is not automatically the same net. Instance connections and bundle unpacking establish the relationship. Conversely, a file may contain multiple modules, so looking only for matching file names can miss the scene unpackers.

27. Build a new game in stages

Treat the following as a development plan for an original implementation. Keep a working version at every stage, and state a pass condition before adding complexity. Choose your own table and rules; reuse the underlying design principles and supplied interface modules appropriately.

Stage 1: prove the physical outputs

Create a small top that exposes the board clock and the OLED connector. Generate a buffered 6.25 MHz clock and connect the provided OLED interface. Feed a constant RGB565 color; then add coordinate-derived color bars. The pass condition is a reliable startup picture, correct colors and the expected corners. Do not start debugging physics while the OLED still shows a blank screen.

Keep VGA independent at first. Generate its proper pixel clock and raster counters, display color bars, and check every active pixel and porch in simulation. The physical pass condition is a stable picture at the intended mode through the converter. A signal on the VGA pins is not enough: the monitor must recognize the clock, sync lengths and polarity.

Stage 2: prove input events and persistent state

Implement a synchronized, debounced button and a one-bit register that toggles on its press event. Observe it on an LED. Hold the button, tap it quickly, and inject simulated bounce. Each accepted press must toggle once. Test two channels together before assuming the filter is independent.

Then implement a small menu state register. A state changes on an action pulse and remains stable between actions. Add reset. At this point you should be able to explain each register's initial value and which exact condition updates it.

Stage 3: draw a movable object

Store one integer or fixed-point coordinate and increment it at a slow enable. Draw a rectangle or ball from that coordinate. It should move at the same speed regardless of display refresh. Pause its position with a control, then resume. Verify bounds before permitting the object to leave the screen.

Add a whole-frame snapshot. Change the live coordinate deliberately halfway through a frame; the current frame should retain its old position, and the next frame should use the new one. This experiment demonstrates tearing and the purpose of a scene register more clearly than merely reading the definition.

Stage 4: make a one-ball physical game

Use signed Q8 X/Y positions and velocities, a fixed update enable and gravity. Add walls, then one bumper, then two flippers and a drain. Separate collision detection from position ownership. Start with axis responses before implementing swept corners. Verify movement numerically in a testbench; a plausible-looking animation can still hide a wrong sign or unit.

Give the controller a launch pulse and the physics engine a final-drain pulse. Implement READY and PLAY first. Add lives and OVER. Every drain must produce exactly one state transition; resetting or preparing during a calculation must cancel stale results. A quick shot should have a minimum usable power, and a released flipper should not sustain play indefinitely.

Stage 5: make scoring reliable

Choose a score representation and a range. Binary arithmetic is simple, but decimal displays require conversion. BCD simplifies four-digit output while requiring decimal carry logic. Define the minimum award unit and how fractions are retained. Derive intermediate widths from the maximum possible transaction, not from a typical hit.

Send contact pulses rather than continuous overlap levels. Add an arming flag and exit hysteresis so a long contact scores once. Test back-to-back awards, saturation, a drain following a hit, and a new game with pending awards. These are module-interaction tests, not just tests of an isolated adder.

Stage 6: add interface quality

Add a score/lives HUD, game-state prompts and charge meter before elaborate decoration. Tie indicators to the authoritative state: protection must come from the same timer used by the save rule, and ball lamps must reflect actual visibility. Keep controls readable in both resolutions.

Then add flash timers, trails and a small particle pool. Draw them behind the active ball and physical objects. Add optional mouse input by translating it into existing game actions. Use one logical control coordinate system for both renderers, with visible hover bounds agreeing with accepted clicks.

Stage 7: add moving geometry and a second ball

Only after single-ball play works, introduce moving object positions and velocities. Use previous and next centres consistently in collision tests; reflecting in the object's velocity frame should transfer motion predictably. Make drawn and physical positions originate from the same stored state.

For two balls, first estimate the cost of a second full collision engine. If it is too large, share one engine using a selector, isolated operand registers, staging and a controller. Test atomic position commits and separately delivered scoring

events. Define whether balls interact physically and whether lives/protection are per ball or per turn. The current project deliberately chooses pass-through balls and turn-wide saves.

Stage 8: implement and measure each substantial addition

Run synthesis and routed implementation while there is still time to change the architecture. A tiny source edit can lengthen a graphics priority chain; a multiplication inside an unrolled loop can duplicate DSPs many times. Simulation cannot tell you the final resource use or path delay.

When the design reaches its target clocks, freeze a version with its bitstream, source hashes, tests and reports. Program exactly that bitstream for the demonstration. Modifying source after the successful build does not modify the FPGA or the old programming file.

28. Verification: prove contracts with self-checking tests

A useful test describes an input sequence and the expected result, event count and timing. A waveform is valuable for diagnosis, but a passing trace that nobody checked is weak evidence. The project uses SystemVerilog testbenches and XSim to reject incorrect behaviour with assertions/fatal checks and to print an explicit PASS marker.

Testbench	Main contract it checks
`tb_input`	Bounce rejection, independent buttons, exact settling, one-cycle events and long-interval counter overflow
`tb_game`	State priorities, triangle charge, fractional/BCD scoring, collections, finite saves, high score and saturation
`tb_physics`	Motion, launch powers/difficulties, speed bounds, moving tracks, pause/cancellation and flippers
`tb_multiball`	Shared engine, atomic pair commits, independent contacts, survivor/final drains and nudges
`tb_two_ball_game`	Real physics-to-rules interactions, pair scoring, carry, saves and last-drain sequencing
`tb_collision`	Contact responses, finite swept corners, near misses, visible guides and flipper underside clearance
`tb_vga`	A complete raster, coordinates, sync widths, RGB conversion and black porches
`tb_vga_clock`	Actual FPGA MMCM simulation lock, 40 MHz and relock after reset
`tb_renderer`	OLED geometry, states, fonts, masks, protection, trails and complete pixel frames
`tb_effects`	Spark origins, bank replacement, trail handoff, pause, lifetime and prepare clearing
`tb_vga_renderer`	Native geometry/UI, pixels, layer priority, fractional positions and complete frames
`tb_top`	Debounced/mouse actions, scene bundle coherence, real protected modules and JC SPI sequencing

The collision tests include both a hit and a nearby trajectory that must miss. Testing only collisions can allow an oversized invisible hit region to pass. The renderer tests similarly check shape edges and points just outside them. Nondefault values in scene tests detect swapped or truncated fields that all-zero fixtures would hide.

The additional `build/final_physics_audit/` oracle compares 18,231 bounded segments against an independent rational geometric calculation. Independence matters: a test that repeats the exact RTL formula can reproduce the same mistake and report success. Random tests complement carefully chosen boundaries; they do not replace them or prove every possible state.

The top-level simulation accelerates long OLED startup waits and board debounce without changing the supplied driver. That makes protocol/state integration practical to test, but it does not measure physical startup delays. Its mouse packet fixtures check the game adapter at the protected controller's outputs, not the behavior of every real USB mouse. Physical tests still check panel startup, cabling, converter mode compatibility and game feel.

Teaching example of a self-check, not a complete testbench:

```
// Drive away from the active sampling edge to avoid a testbench race.
@(negedge clk);
start_press = 1'b1;
@(posedge clk);
#1;  // Observe state after nonblocking updates.
if (state !== READY || !prepare)
    $fatal(1, "Start did not prepare a fresh turn");
```

Simulation-only time delays such as `#1` and `#5` belong in testbenches. They do not create reliable hardware delay in synthesizable game RTL. In the design, counters, clocks and register boundaries represent elapsed time.

29. Vivado flow, constraints and resource reasoning

`FDP2026.xpr` is the saved project. The part and top module must match the board. A module existing on disk does not necessarily mean it is registered in the project's source set. `scripts/verify_project.py` checks registration, part selection, JC mapping, explicit-net guards and protected-driver hashes. `scripts/verify_vivado_project.tcl` checks the saved Vivado project settings and compile order.

`scripts/build.tcl` provides a repeatable batch flow. It reads the mixed VHDL and Verilog sources, reads board constraints, synthesizes the top, reads the implementation-only collision timing constraints after registers are mapped, then optimizes, places, physically optimizes, routes, reports and writes the programming file. It stops if worst setup or hold slack is negative. The board pin check also rejects a stale JA design when this project expects JC.

Understand three different questions. Elaboration asks whether modules, parameters and ports form a legal design. Simulation asks whether the model's behavior matches the expected contract. Synthesis/implementation ask which physical resources implement it and whether signals meet timing after routing. A syntax-correct program can fail any later question.

Setup, hold and why constraints are not a speed-control trick

Setup requires data to arrive before its receiving edge; hold requires it to remain stable around the edge. Worst negative slack means at least one checked path misses its requirement. At 100 MHz, an ordinary single-cycle path has a 10 ns clock period; the native pixel circuit operates against a 25 ns pixel period. Clock uncertainty, skew, setup requirements and routing are included in the actual analysis, so a period is not all available logic time.

Board pins and generated clocks are in `Basys3_Master.xdc`. Asynchronous buttons/switches are exempted only into their first synchronizer stage. Related display clocks remain timed. The scene crossings must settle in the actual edge relationship, which can have only a 5 ns setup window. Putting producer registers before the crossing shortens it by removing derived graphics logic from that path.

The collision block's 16-cycle setup / 15-cycle hold exceptions apply to stable snapshot-to-result paths. They are justified by the scheduler's delayed capture, not by a desire to silence failures. The ordinary selector, control and queued-event paths must still meet normal timing. Changing the capture schedule requires reviewing those exceptions. For the general command semantics, see AMD's UG903 multicycle-path syntax; the project's exact scoped queries are in its XDC file.

Why source-code length is not resource cost

A multiplication can infer a DSP or a substantial LUT circuit. A constant shift may be mostly wiring. A combinational `for` loop normally unrolls into replicated hardware; it is not an economical CPU loop that takes one instruction per iteration. Conversely, a sequential controller with operand multiplexers can reuse one expensive circuit at different times, as the two-ball scheduler does.

A conditional table can infer logic or memory depending on size and style. Calling the font a ROM does not ensure block-RAM use; the measured current design uses zero block-RAM tiles. A full 800 x 600 RGB565 frame would require 7,680,000 bits, so this project avoids a full framebuffer and computes pixels from scene state.

The final routed reports show 88.37% LUTs, 8.56% registers and 60% DSPs. The remaining 11.63% LUT fraction is headroom, not a promise that any feature with a similar-looking source size will fit. Check both area and timing. More utilization is not automatically a better engineering result; extra logic should produce useful behavior, readability or performance.

30. Build, inspect and program this exact project

Open `FDP2026.xpr` in Vivado and use `My_Top_Module` as design top, `tb_top` as the default simulation top. The part is `xc7a35tcpg236-1`. Source files use Verilog, while tests use SystemVerilog and supplied mouse logic uses VHDL. Do not change source languages indiscriminately to work around a warning.

From the project directory in PowerShell, a compact reproducible workflow is:

```
$pinballProject = (Get-Location).Path
$pinballVivado = 'D:\2025.2\Vivado\bin\vivado.bat'
New-Item -ItemType Directory -Force -Path 'build' | Out-Null
& $pinballVivado -mode batch `
  -source (Join-Path $pinballProject 'scripts/build.tcl') `
  -log (Join-Path $pinballProject 'build/finalization.log') `
  -journal (Join-Path $pinballProject 'build/finalization.jou')
& .\scripts\simulate.ps1
python .\scripts\verify_project.py
python .\scripts\render_previews.py
python .\scripts\record_validation.py --build-log finalization.log
```

Adapt the Vivado and Python executable paths to your computer. The simulation script accepts `-VivadoBin`. The preview script requires Pillow; recording uses Python's standard library and checks existing test/build evidence. The current recording script also expects the retained finalization baseline and geometry-audit evidence. It is a provenance recorder for this version, not a substitute for writing new tests when you create a new game.

For an individual regression, use `& .\scripts\simulate.ps1 -Tests tb_game`. For a short selection, pass a PowerShell array such as `-Tests @('tb_input','tb_top')`. The script's serial elaboration setting avoids a host-specific generated-C issue; it does not change the tested hardware behavior. The default GUI simulation runs for 100 microseconds, while some stand-alone benches need far longer. Use Run All or the batch script to reach their explicit PASS conditions.

Successful output goes to `build/pinball_basys3.bit`. In Hardware Manager, open the board target, associate that file with the FPGA and program it. Connect the color Pmod OLEDrgb to JC with the correct orientation. The PROG USB connection to the computer and USB-A mouse host connection have different roles. Allow the provided OLED startup to finish before interpreting a blank picture as a graphics bug.

Do not replace the provided display driver with an assumed monochrome OLED driver: this project expects the 96 x 64 color panel and RGB565. The active converter must accept 800 x 600 VGA. Optional peripherals should be tested separately from core gameplay so a cable problem is not mistaken for a physics error.

JC bus bit	OLED signal	FPGA package pin
0	CS	K17
1	SDIN	M18
2	Unused, driven low	N17
3	SCLK	P18
4	D/C	L17

JC bus bit	OLED signal	FPGA package pin
5	RESN	M19
6	VCCEN	P17
7	PMODEN	R18

31. Debug symptoms by tracing the responsible contract

Symptom	Likely layer and first useful checks
Blank OLED	Clock, startup/reset, JC mapping, power enables, driver pixel request; first test constant color
Monitor reports no signal	MMCM lock, 40 MHz, sync totals/polarity, converter compatibility; first test color bars
Wrong colors	RGB565 channel packing and VGA bit slices
U or menu click repeats	Press event versus held level; debounce or complete-packet edge history
Mouse flipper releases too early	Committed packet buttons versus raw controller button byte
Mouse click affects the wrong row	Dispatch uses previous packet Y or mismatched logical/native hit regions
Ball moves impossibly fast	Position/velocity scale, tick rate, signedness and clamping
Ball sticks or bounces backwards	Penetration correction, approach sign, repeated contact and cooldown
Ball hits an invisible wall	Rendered endpoint versus inflated collision endpoint and finite segment bounds
A corner is crossed without a rebound	Swept segment versus endpoint-only overlap
One collision scores continuously	Event mask validity, per-ball rearm flag, cooldown and exit envelope
Two balls give one award	Masks incorrectly ORed together, event queue spacing or score pipeline
More than three launches	Check finite save timer and turn/life semantics before assuming a life-count bug
Protection color disagrees with drain	Which scene/timer was captured, paused state and final-drain priority
A ghost trail joins two balls	Trail source handoff, clearing history and first-bundle validity mask
HUD flickers or sprite tears	Live values used during a frame instead of the frozen scene
Simulation passes, hardware fails intermittently	Negative routed slack, wrong clock/reset assumptions, unconstrained crossing or stale bitstream

Debug the smallest failing module first. Force or model a known hit transaction to test rules before changing the collision detector. Inspect raw versus filtered contact masks before changing scoring. Capture the pixel value at one suspicious coordinate before rewriting a full renderer. Keep the baseline bitstream/source so you can compare behavior after an intentional change.

Three previous defects make useful examples. An overly broad right-guide X range created an invisible collision beneath a flipper. Painting fresh trail samples before old ones let old pixels dim the head. A fixed-width parameterized input counter wrapped before reaching a long terminal value. Their causes were geometry bounds, layer priority and finite-width state respectively; each needed a different kind of regression.

32. Extending a finalized design without breaking its contracts

Before adding an object, define its logical geometry, its visual geometry, its contact bit or event type, its reward, its arming/exit envelope and any collection memory. Then update collision, physics filtering, rules, renderers and tests coherently. The present 14-bit masks have no spare bit. Expanding them changes arrays, interface widths, bundle widths and test fixtures; stealing a bit from another object silently changes gameplay.

Before changing ball count, estimate state storage and arithmetic separately. A third set of coordinates is cheap compared with a third collision circuit. A shared engine needs another scheduling slot and longer whole-world latency, while pending event/drain spacing and constraint endpoints must remain correct. The current two-ball design does not automatically generalize merely by changing a constant.

Before increasing resolution, derive a supported pixel clock and complete timing mode. Pixel rate determines the budget for computing each output pixel. A 65 MHz mode has a shorter combinational period than 40 MHz even if object geometry is unchanged. Increasing coordinates can also widen multipliers. A display resolution change therefore includes timing, arithmetic widths, layout, constraints and physical converter testing.

Before adding a graphic, determine its final layer and state visibility. A reward popup should not hide a ball or button. A repeated combo notification should not refresh a distracting animation indefinitely. Reuse existing timers and masks when appropriate, but make the behavior intentional rather than dependent on an unrelated field changing by chance.

33. Questions you should be able to answer and explain

1. Which values are state registers, and which are combinational results? Show one of each in the source.
2. Why is the physics tick an enable instead of a clock? Calculate its period from the terminal count.
3. What does raw velocity -128 mean in pixels per step and per second?
4. Why does a moving bumper require both old and next centers, and relative velocity?
5. How can both endpoints of a movement be outside an object even though a collision occurs?
6. What makes the 16-cycle collision timing exception valid, and what change would invalidate it?
7. Why do two contacts on the same object need separate event transactions in two-ball mode?
8. What does pause stop, and what already accepted work is allowed to finish?
9. Why can a protected turn contain more launches than the life counter suggests?
10. What is the difference between binary 123 and BCD `16'h0123`? How is a half-point retained?
11. Which clock edge stores an OLED scene, and which edge advances the supplied SPI controller?
12. Why is a frozen many-bit scene safe here only with the stated related-clock timing assumptions?
13. Why is native VGA more detailed than simply enlarging the OLED image?
14. Why do all-zero bundle tests fail to expose some integration errors?
15. Why can a combinational loop increase resource use, while a sequential scheduler shares one circuit?
16. Which evidence establishes behavior, which establishes physical timing, and which still requires a real board?

If you can explain these answers and implement the staged exercises, you have the foundations to design another interactive FPGA project. The new project's surface appearance may be different, but its engineering still consists of state, arithmetic, events, interfaces, timing and verification.

34. Glossary and source references

Term	Meaning in this guide
RTL	Register-transfer-level description of state and logic between registers
Combinational	Output depends on current inputs; no intended memory
Sequential	Registers retain state across clock edges
FSM	Stored operating state plus transition/output decisions
Datapath	Arithmetic, comparisons, multiplexers and registers operating on values
Clock enable	Condition deciding whether state updates on an existing clock edge
Q8	This project's signed fixed-point values with eight fractional bits; divide by 256
BCD	Four bits per decimal digit; valid digit values 0 through 9
Snapshot	Captured values held as one calculation or display transaction
Atomic commit	Related results become visible together at one clock edge
Hysteresis	Different entry and exit thresholds prevent repeated toggling/contacts
Metastability	A register can take longer to resolve after an asynchronous edge near sampling
LUT	Configurable Boolean lookup logic resource
DSP	Dedicated arithmetic resource in the FPGA
BRAM	Dedicated block memory resource
MMCM / BUFG	Clock synthesis/management primitive and global clock buffer
Setup / hold slack	Margin relative to required data arrival/stability timing
SPI	Serial data interface used to send commands and pixels to the OLED
RGB565 / RGB444	Packed color formats with 5/6/5 and 4/4/4 component bits

The current RTL and its tests are the primary source for the implementation descriptions. Consult `README.md`, `docs/DESIGN_NOTES.md`, `docs/VALIDATION.md` and `build/manifest.json` for the current revision and hardware evidence. Source-code identifiers in this guide can be found directly with text search.

The supplied EE2026 project manual (Academic Year 2026-2027, Semester 1) supports the learning connection to OLED coordinates, RGB565, input actions, smooth movement, anode multiplexing and full integration. The local Lab 4 manual supports the connection to sequential circuits, counters, slower clocks and structural composition. These materials were read as educational source content; their schedules and exercise-specific control mappings do not replace this game's current controls.

The original pinball proposal describes the four components and intended physics/graphics/input features. Its extracted text and rendered pages are retained at workspace `tmp/pdfs/proposal.txt` and `proposal-1.png` through `proposal-5.png`. The original download path was unavailable during guide creation, so the retained extraction and page renders were used rather than inventing proposal details.

For further hardware and constraint reading, use the Digilent Basys 3 manual and AMD UG903 multicycle-path command explanation. Vendor documentation teaches general interface/tool behavior; the project's code and reports establish the particular implementation explained here.

This guide documents a successfully simulated and routed design. No physical board was programmed while creating this guide. It explains the code and its evidence, and identifies the physical checks needed when you reproduce or modify it.